Chapter 1. Prediction Problems and Evaluation Design

Predictive Modeling, Lecture 1 (Is the score trustworthy? Splits, CV, leakage, OLS, logistic regression)

Contents

1 Prediction as a Rule for New Cases **2**

1.1 The prediction problem . . . 2

1.2 The prediction time decides which variables are usable . . . 2

1.3 Regression and classification . . . 2

1.4 Prediction versus causal inference . . . 3

2 Loss, Risk and Generalization **3**

2.1 Loss translates decision costs into numbers . . . 3

2.2 Training score versus deployment risk . . . 4

3 Splitting the Data **4**

3.1 Three roles . . . 4

3.2 The split must preserve the data-generating structure . . . 5

3.3 Reproducibility . . . 5

4 Information Leakage **5**

5 K-fold Cross-Validation **6**

6 Metrics are the Language of Decisions **7**

6.1 Regression metrics . . . 7

7 Classification: Probabilities, Thresholds, Metrics **7**

7.1 Probability first, action second . . . 7

7.2 Threshold and confusion matrix . . . 7

7.3 Three questions about one probability . . . 8

7.4 Choosing the threshold from costs . . . 8

8 The Linear Baseline: OLS **9**

8.1 Least squares . . . 9

8.2 Predictive versus causal reading of coefficients . . . 9

8.3 Residual diagnostics point to the next model . . . 9

9 Logistic Regression **9**

9.1 From linear score to probability . . . 9

9.2 Log loss . . . 10

10 Comparing Candidates Fairly **10**

Roadmap of the predictive-modeling part

Lecture	Key question	Models and concepts
1	Can the score be trusted?	splits, CV, leakage, OLS, logistic regression
2	Many correlated variables?	Ridge, Lasso, Elastic Net
3	How to learn nonlinear relations?	decision trees, bagging, random forest
4	Strong models for tabular data?	GBM, XGBoost, LightGBM, CatBoost
5	When are neural nets useful?	MLP, minibatch, Adam, regularization, TabPFN/TabICL
6	How to combine many models?	AutoGluon, OOF, bagging, stacking

The guiding principle of the whole part: **models may change, but the prediction time, the split, the pipeline and the final-test rule must not.**

1 Prediction as a Rule for New Cases

1.1 The prediction problem

Definition: prediction rule

From observed data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^p$ we build a rule

$$\hat{f} : \mathcal{X} \to \mathcal{Y}, \qquad \hat{y}_{\text{new}} = \hat{f}(x_{\text{new}}).$$

x_i is the feature vector already observed at the **prediction time**, y_i is the target observed later, n is the number of cases and p the number of features. The object of interest is the **prediction error on new cases**, not the fit to $\mathcal{D}$.

A predictive model does not try to explain every relation in the data; it is a rule that reduces future uncertainty *using only information available at prediction time.*

1.2 The prediction time decides which variables are usable

Every prediction problem must state *when* the prediction is made. If we predict next month's churn at the end of this month, nothing that happens after the end of the month may be a feature.

Role of a column	Question at prediction time	Churn example
Feature X_j	Is it already known at that moment?	recent usage, contract length, # complaints
Target Y	What do we predict?	churn next month
Identifier (ID)	Does it carry anything beyond row identity?	customer ID, account serial number
Post-event variable	Does it arise only after the outcome?	reason for cancellation, refund after cancellation

Post-event variables may raise the score dramatically, but they do not exist at the real prediction time and must not be used.

1.3 Regression and classification

Task	Target	Direct model output	Focus of evaluation
Regression	continuous Y	$\hat{y}(x)$, conditional mean	average error and large errors
Binary classification	$Y \in \{0, 1\}$	$\hat{p}(x) = P(Y = 1 \mid X = x)$	ranking, probability, threshold errors
Multiclass	$Y \in \{1, \dots, C\}$	$(\hat{p}_1, \dots, \hat{p}_C)$	per-class errors and probability quality

In classification the model first produces a **probability, not a label**; turning that probability into an action is a separate decision (§7).

1.4 Prediction versus causal inference

Prediction: $\hat{y} = \hat{f}(x)$	Causal inference: $\mathbb{E}\{Y(1) - Y(0)\}$
Who is likely to churn?	How much did a coupon reduce churn?
Small loss on new cases is the goal.	Identification assumptions and bias control are central.
Correlated (non-causal) variables can be useful features.	Confounders and post-treatment variables need care.

A good predictive coefficient alone never justifies a causal or policy claim.

2 Loss, Risk and Generalization

2.1 Loss translates decision costs into numbers

Definition: loss and risk

A loss $L(y, f(x)) \geq 0$ measures the cost of predicting $f(x)$ when the truth is y. The **risk** in the deployment environment is

$$R(f) = \mathbb{E}_{(X,Y)\sim P_{\text{deploy}}}[L\{Y, f(X)\}].$$

$R(f)$ cannot be observed directly. The **training risk** is $\widehat{R}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L\{y_i, f(x_i)\}$, and the **generalization gap** is $\text{Gap}(f) = R(f) - \widehat{R}_{\text{train}}(f)$.

The loss should be fixed *before* the model: overstock and stock-out have different costs; missing a defaulter and rejecting a good customer have different costs; if bids are set from probabilities then probability quality matters more than labels. Accuracy is just one particular loss, not the default goal of every problem.

Theorem: what each loss targets

Let $f^\star = \arg\min_f R(f)$ over all measurable f. Pointwise in x:

1. squared loss $(y - f)^2$: $f^\star(x) = \mathbb{E}[Y \mid X = x]$ (conditional mean);
2. absolute loss $|y - f|$: $f^\star(x) = \text{median}(Y \mid X = x)$;
3. quantile (pinball) loss $\rho_\tau(y - f)$, $\rho_\tau(u) = u(\tau - \mathbb{I}\{u < 0\})$: $f^\star(x) =$ the conditional τ-quantile;
4. 0–1 loss for $Y \in \{0, 1\}$: $f^\star(x) = \mathbb{I}\{P(Y = 1 \mid x) \geq 1/2\}$ (Bayes classifier);

5. log loss $-[y \log q + (1-y)\log(1-q)]$ over $q \in (0,1)$: $q^\star(x) = P(Y=1 \mid X=x)$ (a *proper* scoring rule).

Proof. By the tower property $R(f) = \mathbb{E}_X[\mathbb{E}\{L(Y, f(X)) \mid X\}]$, so it suffices to minimize $g(c) = \mathbb{E}[L(Y,c) \mid X = x]$ in c for each x.

(1) $g(c) = \mathrm{Var}(Y \mid x) + (\mathbb{E}[Y \mid x] - c)^2$. (2)–(3) $g'(c) = -\tau + P(Y < c \mid x)$ (right derivative), which is ≤ 0 for c below the τ-quantile and ≥ 0 above; $\tau = \frac{1}{2}$ gives the median.

(4) $g(1) = 1 - \pi(x)$, $g(0) = \pi(x)$ with $\pi(x) = P(Y = 1 \mid x)$; choose 1 iff $\pi \ge \frac{1}{2}$.

(5) $g(q) = -\pi \log q - (1-\pi)\log(1-q)$, $g'(q) = -\pi/q + (1-\pi)/(1-q) = 0 \iff q = \pi$, and $g'' > 0$. ■

2.2 Training score versus deployment risk

Learning algorithms fit by minimizing $\widehat{R}_{\text{train}}$, and complex models can drive it near zero. What we want is $R(f)$, the average loss on unseen cases, which the training loss cannot reveal. Hence an **independent validation procedure** is needed. **Generalization** is the property that a rule learned on training data keeps working on new cases.

Typical causes of an unstable generalization gap: searching over too many candidate models and hyperparameters; random splits that ignore time or group structure; preprocessing and variable selection done on the whole data; distribution shift between training and deployment.

Theorem: bias–variance decomposition

Let $Y = f(x) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\mathrm{Var}\varepsilon = \sigma^2$, and let $\hat{f}$ be fitted on a random training set independent of the new (x, Y). Then

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \underbrace{\left(f(x) - \mathbb{E}\hat{f}(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathrm{Var}\hat{f}(x)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Proof. Write $Y - \hat{f} = \varepsilon + (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f})$. The three terms are pairwise uncorrelated: ε is independent of $\hat{f}$ and has mean zero; $f - \mathbb{E}\hat{f}$ is a constant and $\mathbb{E}(\mathbb{E}\hat{f} - \hat{f}) = 0$. Squaring and taking expectations gives the result. ■

Model complexity moves the two curves differently: training error decreases monotonically, test error is U-shaped. **Underfitting** misses the signal (high bias), the right complexity balances signal and noise, **overfitting** memorizes accidental wiggles (high variance).

3 Splitting the Data

3.1 Three roles

Part	Used for	Never used for
Training part	fitting model parameters	—
Internal validation (holdout or CV)	choosing candidates and tuning	—
Test set	reporting final generalization once	choosing models

The test set is not material for the next choice; it is the *conclusion* of this experiment.

3.2 The split must preserve the data-generating structure

Data structure	Recommended split	Danger of a random split
independent rows, class imbalance	stratified split	minority-class proportion may differ by chance
clustered (customers, hospitals, households)	group split	the same group appears in training and test
time order	forward (time-ordered) split	future information leaks into training
duplicates / near-duplicates	deduplicate, then split	nearly identical rows on both sides

If we train on 2024 and predict 2025, the order past → future must be kept. Several transactions of one customer must be split at the customer level. Unless the *unit of a new case* is defined, the correct validation split cannot be defined either.

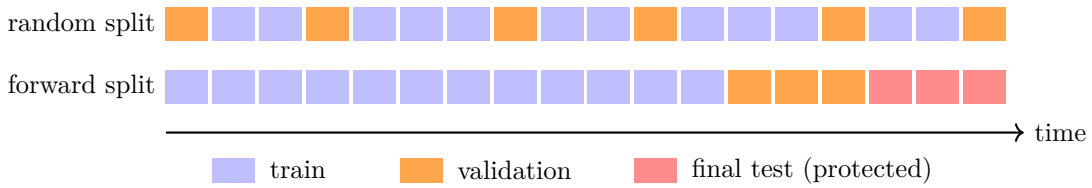


Figure 1: With time-ordered data a random split lets the model train on the future of its validation cases; a forward split trains on the past, validates on the next period and keeps the latest period untouched as the final test. Group splits follow the same idea with customers or hospitals in place of time blocks.

3.3 Reproducibility

Fix: split rule and random seed; feature list and target definition; handling of missing values, categories, scaling; code computing metrics and thresholds.

Record: extraction time, data version, number of rows; class ratios and number of groups in each set; candidate models and hyperparameter grids; fold-by-fold variability, not only the CV mean.

Reproducibility means not only getting the same numbers again but being able to *trace why numbers changed*.

4 Information Leakage

Definition: leakage

Leakage is any channel through which information unavailable at prediction time, or information from the validation/test part, enters the fitting procedure.

Type	Example	Why it is a problem
Target leakage	cancellation reason or default-handling result used as a feature	does not exist at prediction time
Preprocessing leakage	mean imputation / standardization on the full data	peeks at the validation/test distribution
Temporal leakage	future averages, after-the-fact aggregates	cannot reproduce a past decision
Duplicate / group leakage	same customer, document or product on both sides	the "new" case was effectively already seen

When performance is suspiciously high, suspect leakage before praising the algorithm.

Example: why preprocessing must be learned inside each fold

Standardizing with the full-data mean $\bar{x}$ and standard deviation s uses validation rows to build the training features. With p large and screening ("keep the 20 features most correlated with y") done on the full data, a model trained on pure noise can show a high CV score, because the screening step has already looked at the validation targets. The fix is to put every data-dependent step (imputation, scaling, screening, encoding) inside the pipeline that is re-fitted on the training part of each fold.

5 K-fold Cross-Validation

Definition: K-fold CV

Split the development data into folds $V_1, \ldots, V_K$. For each k fit $\hat{f}^{(-k)}$ on the data without V_k and compute

$$\widehat{R}_k(f) = \frac{1}{|V_k|} \sum_{i \in V_k} L\{y_i, \hat{f}^{(-k)}(x_i)\}, \qquad \widehat{R}_{\mathrm{CV}}(f) = \frac{1}{K} \sum_{k=1}^{K} \widehat{R}_k(f).$$

Each observation is used once for validation and $K-1$ times for training. Comparison of candidates is more stable than with a single validation set.

Remark: fold variability and its standard error

Report $\widehat{R}_{\mathrm{CV}}$ with $\mathrm{SE} = \mathrm{sd}(\widehat{R}_1, \ldots, \widehat{R}_K)/\sqrt{K}$. The folds share training data, so this SE is only a rough guide (it usually underestimates the true uncertainty), but it is what the 1-SE rule of Ch. 3 uses.

Algorithm: selection with CV, reporting with the test set

1. Split off the test set first and lock it away.
2. On the remaining development data, define the pipeline (preprocessing + candidate model).
3. Choose candidates and hyperparameters using CV scores and fold variability.
4. Refit the chosen pipeline on the whole development data.
5. Report final performance, error patterns and operating cost on the test set.

6 Metrics are the Language of Decisions

A good metric moves in the same direction as operational loss, is used consistently across candidates, does not hide imbalance or threshold effects, and can be explained to stakeholders. One model needs several metrics: for regression, average error, large errors, errors by segment; for classification, ranking, probability quality, errors at a threshold; and the final choice weighs metric, stability, computing time and explainability. Ask first *which errors are actually more expensive.*

6.1 Regression metrics

Metric	Definition	Interpretation
MAE	$\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i\rvert$	all errors linearly; linked to median prediction
MSE	$\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$	squared penalty on large errors; linked to mean prediction
RMSE	$\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$	same unit as y; emphasizes large errors
R^2	$1 - \frac{\sum_i (y_i-\hat{y}_i)^2}{\sum_i (y_i-\bar{y})^2}$	improvement in squared error over the mean baseline

Theorem: $\mathrm{MAE} \le \mathrm{RMSE} \le \sqrt{n}\,\mathrm{MAE}$

For residuals $e_i = y_i - \hat{y}_i$, $\frac{1}{n}\sum |e_i| \le \left(\frac{1}{n}\sum e_i^2\right)^{1/2}$, with equality iff all $|e_i|$ are equal.

Proof. Cauchy–Schwarz: $\left(\sum |e_i| \cdot 1\right)^2 \le n \sum e_i^2$. The upper bound follows from $\sum e_i^2 \le (\sum |e_i|)^2$. ■

The gap RMSE − MAE grows with the dispersion of $|e_i|$: a smaller RMSE means that large errors in particular were reduced. Prefer RMSE when rare large errors are especially costly or when models minimizing squared loss are compared; prefer MAE when the influence of outlying targets should be limited or the typical error should be communicated. If the two metrics rank two models differently, the models reduced errors of different *size and distribution.*

7 Classification: Probabilities, Thresholds, Metrics

7.1 Probability first, action second

A classifier first outputs $\hat{p}(x) = P(Y = 1 \mid X = x)$, an estimate of a conditional probability. The same probability can lead to different actions under different business rules (churn probability $\hat{p} = 0.18$: early contact if coupons are cheap; only the top 5% if counselling staff is limited; a lower threshold if losing the customer is very costly). One can also check whether the group predicted at 0.18 actually churns at about 18% (calibration).

7.2 Threshold and confusion matrix

$$\hat{y}_t(x) = \mathbb{I}\{\hat{p}(x) \ge t\},$$

	actual +	actual −
predicted +	TP	FP
predicted −	FN	TN

$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}, \qquad \text{accuracy} = \frac{TP+TN}{n}.$$

Lowering t produces more positive predictions: recall usually rises but false positives rise too.

Example: accuracy under imbalance

Of 1,000 customers only 50 churn. Predicting "no churn" for everyone gives accuracy = 95%, yet finds no churner (recall 0). With imbalance, look at precision (quality of the selected group), recall (share of positives found) and the costs together.

7.3 Three questions about one probability

Ranking	Reliability of the probability	Action
Whom to look at first? ROC-AUC, PR-AUC	Does 0.30 mean about 30%? Brier score, log loss, calibration plot	Where to place the threshold? precision/recall at t, cost

A good ranking does not imply well-calibrated probabilities, nor fitness for the current operating rule.

Theorem: ROC-AUC is a ranking probability

For a score s, $\text{AUC} = P(s(X^+) > s(X^-)) + \frac{1}{2}P(s(X^+) = s(X^-))$, where X^+, X^- are independent draws from the positive and negative classes. Empirically AUC equals the proportion of (positive, negative) pairs ranked correctly (Mann–Whitney statistic).

Proof. The ROC curve plots $\text{TPR}(t) = P(s^+ > t)$ against $\text{FPR}(t) = P(s^- > t)$. Parametrize by t and integrate: $\int \text{TPR}\, d\text{FPR} = \int P(s^+ > t)\, dF^-(t) = P(s^+ > s^-)$ for continuous scores (ties contribute half). ■

PR-AUC is preferred when positives are rare, since it focuses on precision and recall. Brier score $\frac{1}{n}\sum(y_i - \hat{p}_i)^2$ is the mean squared probability error; log loss heavily penalizes confident wrong probabilities. AUC is invariant to any monotone transformation of the scores, so it says nothing about calibration.

7.4 Choosing the threshold from costs

Theorem: cost-optimal threshold

Let a false negative cost C_{FN} and a false positive cost C_{FP} (correct decisions cost 0). If $\hat{p}(x)$ is calibrated, acting positive minimizes expected cost iff

$$\hat{p}(x)\, C_{FN} > \{1 - \hat{p}(x)\}\, C_{FP} \iff \hat{p}(x) > t^\star = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Proof. Acting positive costs C_{FP} with probability $1-\hat{p}$; acting negative costs C_{FN} with probability $\hat{p}$. Compare the two expected costs and rearrange. ■

As C_{FN} grows the threshold for acting positive *decreases*; $t^\star = 1/2$ only when the costs are equal. The prediction of a probability and the action rule should be thought of separately.

8 The Linear Baseline: OLS

8.1 Least squares

$$\hat{y}_i = \beta_0 + x_i^\top \beta, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \beta_0 - x_i^\top \beta)^2.$$

With the design matrix X including the intercept column,

Derivation: normal equations

$Q(\beta) = \|y - X\beta\|^2 = y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta$, so $\nabla Q = -2X^\top y + 2X^\top X\beta = 0$ gives $X^\top X\hat{\beta} = X^\top y$. If $X^\top X$ is invertible, $\hat{\beta} = (X^\top X)^{-1} X^\top y$; the Hessian $2X^\top X \succeq 0$ shows it is a global minimum. Geometrically $\hat{y} = X\hat{\beta} = Hy$, $H = X(X^\top X)^{-1}X^\top$, is the orthogonal projection of y onto the column space, and the residual is orthogonal to every column: $X^\top(y - \hat{y}) = 0$.

If $X^\top X$ is unstable (nearly singular) the coefficient estimates are unstable too; regularization (Ch. 3) mitigates this.

8.2 Predictive versus causal reading of coefficients

Predictively, β_j is the change of the linear prediction when X_j increases by one unit with other features held fixed. Under multicollinearity the sign and size of coefficients can wobble, and coefficients of variables in different units are not comparable. A causal reading additionally requires control of confounders and a correct causal structure, exclusion of post-treatment variables, and checks for selection bias and measurement error.

8.3 Residual diagnostics point to the next model

Observed phenomenon	Meaning	Next candidates
curved residual pattern	missed nonlinearity / interaction	transformations, splines, trees (Ch. 4)
variance changing with $\hat{y}$	heteroscedasticity, scale issue	transformations, robust loss, segment-wise evaluation
a few huge residuals	outliers, label errors	Huber loss, data checks (Ch. 2)
unstable coefficients with correlated features	multicollinearity, high dimension	Ridge, Lasso, Elastic Net (Ch. 3)

Seeing where the linear baseline fails shows *why* the models of the following lectures are needed.

9 Logistic Regression

9.1 From linear score to probability

$$\eta(x) = \beta_0 + x^\top \beta, \qquad \hat{p}(x) = \sigma\{\eta(x)\} = \frac{1}{1 + \exp\{-\eta(x)\}}, \qquad \log \frac{p(x)}{1 - p(x)} = \beta_0 + x^\top \beta.$$

The linear score lives on the whole real line; the sigmoid squashes it into $(0, 1)$. β_j is the change of the **log-odds** when X_j increases by one unit with the others fixed, $\exp(\beta_j)$ is the **odds ratio**, and the change in the probability itself depends on the current x: $\partial p / \partial x_j = \beta_j p(1 - p)$.

9.2 Log loss

$$\ell(y,p) = -\big[y\log p + (1-y)\log(1-p)\big], \qquad \hat\beta = \arg\min_\beta \sum_{i=1}^n \ell\{y_i, \sigma(\beta_0 + x_i^\top \beta)\}.$$

A positive case given $p \approx 0$, or a negative case given $p \approx 1$, receives a huge penalty: log loss evaluates the *quality of probabilities*, not just right/wrong labels.

Theorem: gradient and convexity of the logistic log loss

With $p_i = \sigma(x_i^\top \beta)$ (intercept absorbed), $J(\beta) = \sum_i \ell(y_i, p_i)$ satisfies

$$\nabla J(\beta) = X^\top (p - y), \qquad \nabla^2 J(\beta) = X^\top W X, \quad W = \mathrm{diag}\{p_i(1-p_i)\} \succeq 0,$$

so J is convex and minimizing log loss equals maximum likelihood for the Bernoulli model. Newton's method is iteratively reweighted least squares.

Proof. $\sigma'(\eta) = \sigma(\eta)\{1-\sigma(\eta)\}$, so $\partial\ell/\partial\eta = -y(1-p) + (1-y)p = p - y$. The chain rule gives $\nabla J = \sum_i (p_i - y_i)x_i$, and differentiating once more gives $\sum_i p_i(1-p_i)x_i x_i^\top$. The Bernoulli log-likelihood is $\sum_i \{y_i \log p_i + (1-y_i)\log(1-p_i)\} = -J$. ■

Remark: separation

If a hyperplane separates the two classes perfectly, J keeps decreasing as $\|\beta\| \to \infty$ and the MLE does not exist; penalization (Ch. 3) restores a finite solution.

10 Comparing Candidates Fairly

Criterion	Conditions to control
same split	all candidates use the same training/CV/test structure
same feature definition	same prediction time, missing-value and category handling
same metric	primary and secondary metrics for selection fixed in advance
same compute budget	record search range, time and number of iterations
same reporting	show CV mean, variability, test score, complexity and operating constraints together

Model comparison is not a popularity contest between algorithms but a **controlled experiment**.

Summary

- Define the target and the prediction time first.
- The goal is low generalization risk in deployment, not low training error.
- The split must preserve the data-generating structure: time, groups, duplicates.
- CV is for selection, the test set for the final report; preprocessing is learned inside each training fold.
- Metrics, thresholds and baselines are chosen from decision costs before algorithms.